

Ampliando: MCP para creadores de contenido

Tipos de servidores MCP y cómo usarlos

JA Palazón Ferrando

2026-06-21

Contenidos

1	¿Qué es MCP?	2
2	Tipos de servidores MCP	2
3	Servidores de búsqueda y web	3
3.1	Fetch (mcp-fetch)	3
3.2	Brave Search	3
3.3	DuckDuckGo Search	3
3.3.1	Ejemplo práctico: información actualizada sobre educación en China	4
3.4	Firecrawl	5
4	Automatización de navegadores	5
4.1	Playwright	5
5	Documentación y conocimiento	5
5.1	Context7	5
5.2	Memory (Mem0)	5
6	Razonamiento	6
6.1	Sequential Thinking	6
7	Búsqueda de código	6
7.1	Grep	6
7.2	Filesystem	6
8	Servidores de contenido	6
8.1	Quarto MCP	6
8.2	Quarto Book MCP	7
9	Aprendizaje y educación	7
9.1	Flashcards MCP	7
10	Ejercicios prácticos	8
	Ejercicio 1	8
	Ejercicio 2	8
	Ejercicio 3	8
	Ejercicio 4	8
11	Configuración general	9
11.1	Estructura de opencode.json	9

11.2 Niveles de configuración 9

11.3 Habilitar/deshabilitar por agente 9

12 Buenas prácticas 10

13 Otros servidores MCP disponibles 10

14 Referencias 10

i Has visto lo básico en el paso 5

Este documento amplía lo que viste en el paso 5. Si no lo has visto aún, te recomiendo empezar por allí.

💡 ¿Para quién es esta guía?

Para creadores de contenido que quieren conectar OpenCode con fuentes externas de información: glosarios, wikis, bases de datos y otros servicios. **No necesitas saber programar** para entenderla.

1. ¿Qué es MCP?

El **Model Context Protocol (MCP)** es un estándar abierto que permite conectar modelos de lenguaje con herramientas externas. Creado por Anthropic en noviembre de 2024, MCP resuelve el problema de integraciones: en lugar de crear código personalizado para cada par de modelo-herramienta, MCP proporciona una interfaz universal.

¿Por qué MCP?

- **Estándar único:** un servidor MCP funciona con Claude, ChatGPT, Cursor, VS Code y otros clientes compatibles
- **Ecosistema creciente:** más de 19.000 servidores MCP registrados (2026)
- **Gobernanza abierta:** donado a la Linux Foundation en diciembre de 2025

Analogía: piensa en MCP como una biblioteca personal que OpenCode puede consultar cuando lo necesita. En lugar de darte la respuesta de memoria, busca en fuentes reales y actualizadas.

2. Tipos de servidores MCP

Tipo	Ejemplo	Cuándo usarlo
Remoto	GitHub, Notion, Brave Search	Servicios en la nube
Local	filesystem, sqlite	Datos en tu ordenador
Desarrollado por ti	Tu propio servidor	Necesidades específicas

Los servidores locales son ideales para datos sensibles que no quieres subir a la nube. Los remotos son más fáciles de configurar y no requieren instalación.

3. Servidores de búsqueda y web

3.1. Fetch (mcp-fetch)

Función: descargar contenido de páginas web. Como copiar y pegar el contenido de una web, pero automático. OpenCode puede usarlo para leer artículos, descargar PDFs o acceder a datos de *APIs* (interfaces que permiten consultar servicios web).

Ejemplos:

- *Descarga el contenido de esta página web y resúmelo*
- *Lee este artículo y extrae los puntos clave*

Configuración:

```
{
  "mcp": {
    "fetch": {
      "type": "remote",
      "url": "https://mcp.fetch.ai"
    }
  }
}
```

3.2. Brave Search

Función: buscador web como Google, pero diseñado para herramientas de IA. Permite buscar información actualizada, filtrar por fecha y región, y obtener resúmenes de las páginas encontradas. Útil cuando necesitas datos recientes que el modelo de IA no conoce.

Ejemplos:

- *Busca las últimas novedades sobre inteligencia artificial en educación*
- *¿Qué se dice de OpenCode en foros de programadores?*

3.3. DuckDuckGo Search

Función: búsqueda web con protección de privacidad. DuckDuckGo no rastrea al usuario ni crea perfiles de navegación, a diferencia de otros buscadores.

Aplicaciones:

- Búsqueda de información actualizada sin comprometer la privacidad
- Investigación de temas específicos con filtros por región
- Búsqueda de noticias recientes sobre un tema
- Obtención de contenido de páginas web específicas

Las tres herramientas del MCP:

Herramienta	Función	Parámetros principales
search	Búsqueda web general	query, max_results, region
fetch_content	Obtener contenido de una URL	url, backend
news_search	Búsqueda de noticias recientes	query, max_results, time_filter

Nota: el `fetch` ya viene integrado en OpenCode. El `fetch_content` de DuckDuckGo es una alternativa con protección de privacidad.

Configuración en `opencode.json`:

```
{
  "mcp": {
    "duckduckgo": {
      "type": "local",
      "command": ["uvx", "duckduckgo-mcp-server"]
    }
  }
}
```

3.3.1. Ejemplo práctico: información actualizada sobre educación en China

Los modelos de IA tienen conocimientos hasta una fecha de corte determinada. Cuando necesitas información reciente, los MCP de búsqueda son esenciales. Este ejemplo muestra cómo usar las tres herramientas de DuckDuckGo para investigar la reducción de estudios de grado en China.

Paso 1 — Búsqueda general con `search`:

En *Plan*, escribe:

Busca información sobre la reducción de carreras universitarias en China en 2026

OpenCode usará la herramienta `search` para encontrar artículos relevantes. Entre los resultados aparecerán fuentes como Vandal, Substack y medios especializados en tecnología y educación.

Paso 2 — Búsqueda de noticias recientes con `news_search`:

Para obtener solo noticias del último mes:

Busca noticias recientes sobre universidades chinas eliminando carreras de arte

La herramienta `news_search` filtra por fecha y devuelve resultados con fuente y fecha de publicación. Así puedes verificar que la información es actual y no obsoleta.

Paso 3 — Obtener contenido completo con `fetch_content`:

Cuando encuentres un artículo relevante, puedes extraer su contenido:

Extrae el contenido completo de este artículo sobre educación en China

La herramienta `fetch_content` descarga y parsea la página, devolviendo texto limpio para su análisis. Útil para verificar datos antes de incluirlos en un curso o artículo.

Sugerencias de uso:

Situación	Herramienta recomendada
Investigar un tema actualizado	<code>search</code> con filtros de región
Encontrar noticias de la semana	<code>news_search</code> con <code>time_filter</code>
Verificar datos de una fuente	<code>fetch_content</code> con la URL
Buscar en otro idioma	<code>search</code> con <code>region</code> (ej: <code>cn-zh</code>)

3.4. Firecrawl

Función: exploración web inteligente. A diferencia de Fetch que descarga una sola página, Firecrawl puede recorrer un sitio web completo (*crawling*), extraer tablas, listas y contenido organizado (*scraping*), y convertir el HTML a texto limpio.

Ejemplos:

- *Extrae todas las entradas de este blog sobre Quarto*
- *Descarga los precios de esta tienda online y guárdalos en un dataset*

4. Automatización de navegadores

4.1. Playwright

Función: control remoto de un navegador web. Puede abrir Chrome, Firefox o Safari, navegar por páginas, hacer clic en botones, rellenar formularios y capturar pantallas. Como tener un asistente que usa el navegador por ti. El *testing E2E* (pruebas de extremo a extremo) significa que puede simular un usuario real completando todo un proceso, desde abrir la web hasta finalizar una compra.

Ejemplos:

- *Abre Google y busca ‘curso de Python’*
- *Rellena este formulario con mis datos y envíalo*

5. Documentación y conocimiento

5.1. Context7

Función: documentación técnica actualizada. Las *librerías* son conjuntos de código reutilizable (como plantillas), y los *frameworks* son estructuras que facilitan el desarrollo. Context7 permite a OpenCode consultar la documentación más reciente de estas herramientas, en lugar de usar conocimientos que pueden estar desactualizados.

Ejemplos:

- *¿Cómo se usa la función `filter()` en Python?*
- *¿Cuál es la sintaxis actual de Quarto para tablas?*

5.2. Memory (Mem0)

Función: memoria a largo plazo para OpenCode. Sin Memory, cada conversación empieza desde cero. Con Memory, OpenCode recuerda preferencias, decisiones anteriores y contexto importante entre sesiones. Usa un *grafo de conocimiento*, que organiza información relacionando conceptos entre sí, como un mapa mental que conecta ideas.

Ejemplos:

- *Recuerda que siempre escribo en español y prefiero explicaciones breves*
- *¿Qué decidimos en la conversación anterior sobre el diseño del curso?*

6. Razonamiento

6.1. Sequential Thinking

Función: pensamiento paso a paso. Cuando un problema es complejo, ayuda a OpenCode a descomponerlo en pasos lógicos, analizar opciones y llegar a una conclusión fundamentada. Cada paso se construye sobre el anterior, como resolver un problema de matemáticas mostrando todos los pasos.

Ejemplos:

- *Analiza pros y contras de usar Quarto vs Markdown puro*
- *¿Cuál es la mejor estructura para este curso? Piensa paso a paso*

7. Búsqueda de código

7.1. Grep

Función: búsqueda avanzada de texto. Como usar Ctrl+F, pero mucho más potente. Puede buscar patrones complejos en archivos. Las *expresiones regulares* son un lenguaje de búsqueda que permite definir patrones flexibles, como buscar todas las palabras que tengan exactamente 5 letras o que empiecen por “get_”.

Ejemplos:

- *Busca en todos los archivos .qmd las líneas que contengan ‘callout’*
- *¿En qué archivos se usa la palabra ‘repetición espaciada’?*

7.2. Filesystem

Función: operaciones de archivos y directorios.

Aplicaciones:

- Leer y escribir archivos
- Explorar estructura de directorios
- Copiar, mover y eliminar archivos
- Búsqueda de archivos por patrones

Seguridad:

- Acceso restringido a directorios específicos
- Permisos configurables
- Auditoría de operaciones

8. Servidores de contenido

8.1. Quarto MCP

Función: convertir documentos Quarto Markdown (.qmd) a múltiples formatos de salida, con enfoque principal en presentaciones PowerPoint.

Características principales:

- **PowerPoint como formato principal:** diseñado para creación de presentaciones empresariales
- **Plantillas personalizadas:** soporte para plantillas .pptx corporativas
- **Validación Mermaid:** detección de errores en diagramas antes de renderizar
- **Multi-formato:** PDF, HTML, Word, Reveal.js, EPUB, LaTeX y más
- **Seguridad:** sin ejecución de código (`--no-execute` fijo)

Instalación:

```
# Desde repositorio Git (recomendado)
uvx --from git+https://github.com/notfolder/quarto_mcp quarto-mcp
```

Configuración en opencode.json:

```
{
  "mcp": {
    "quarto": {
      "type": "local",
      "command": ["uvx", "--from", "git+https://github.com/notfolder/quarto_mcp", "quarto-mcp"]
    }
  }
}
```

8.2. Quarto Book MCP

Función: MCP minimalista para proyectos Quarto Book que procesa reglas de estilo y formato.

Características:

- Procesa `rules-catalog.json` generado por Quarto Book
- Aplica reglas de formato y estilo de forma automática
- Diseñado para libros técnicos y documentación

9. Aprendizaje y educación

9.1. Flashcards MCP

Función: crear, gestionar y estudiar tarjetas de aprendizaje (*flashcards*) con repetición espaciada y generación con IA. El *algoritmo FSRS* (Free Spaced Repetition Scheduler) calcula cuándo necesitas repasar cada tarjeta según tu nivel de dificultad. Las tarjetas fáciles aparecen menos veces, las difíciles más. Como un profesor que sabe exactamente cuándo recordarte cada concepto.

Ejemplos:

- *Crea flashcards sobre los tipos de MCP que hemos visto*
- *¿Qué flashcards tengo pendientes de repasar hoy?*

Configuración en opencode.json (servidor remoto):

```
{
  "mcp": {
    "flashcards": {
      "type": "remote",
      "url": "https://flashcards.louisarge.com/api/mcp"
    }
  }
}
```

Nota: este MCP es remoto, no necesita instalación local. Solo configurar la URL. Para más detalles, consulta la guía de ampliación.

10. Ejercicios prácticos

Ejercicio 1

Antes de configurar servidores MCP, necesitamos un directorio de trabajo con contenido. Crea un directorio llamado `glosario` dentro de Documentos y guarda un archivo llamado `glosario.md` con este contenido:

```
# Glosario de términos

**SLA**: Acuerdo de Nivel de Servicio (*Service Level Agreement*).
Compromiso formal entre un proveedor y un cliente sobre la calidad
del servicio.

**MCP**: Protocolo de Contexto de Modelo (*Model Context Protocol*).
Estándar abierto para conectar modelos de IA con herramientas externas.

**API**: Interfaz de Programación de Aplicaciones (*Application Programming Interface*).
Forma en que dos aplicaciones se comunican entre sí.
```

Ejercicio 2

El servidor de sistema de archivos permite a OpenCode leer archivos de directorios concretos de forma segura.

Crea un archivo `.opencode/mcp.json` en la raíz del proyecto:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/ruta/a/tu/directorio/compartido"
      ]
    }
  }
}
```

Cambia `/ruta/a/tu/directorio/compartido` por un directorio real de tu sistema, por ejemplo `/home/usuario/Documentos/glosario`

Ejercicio 3

Cierra y vuelve a abrir OpenCode para que cargue la nueva configuración. Luego en *Plan*:

Busca en el glosario el término SLA y explícame qué significa

OpenCode usará el servidor MCP de archivos para leer el glosario y responderte con la definición.

Ejercicio 4

Los servidores MCP también pueden conectarse a bases de datos. Añade un servidor SQLite al `mcp.json`:

```
{
  "mcpServers": {
    "filesystem": {
```

```

    "command": "npx",
    "args": [
      "-y",
      "@modelcontextprotocol/server-filesystem",
      "/home/usuario/Documentos/glosario"
    ]
  },
  "sqlite": {
    "command": "npx",
    "args": [
      "-y",
      "@modelcontextprotocol/server-sqlite",
      "/home/usuario/Documentos/glosario.db"
    ]
  }
}

```

11. Configuración general

11.1. Estructura de opencode.json

```

{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "fetch": {
      "type": "remote",
      "url": "https://mcp.fetch.ai"
    },
    "brave-search": {
      "type": "remote",
      "url": "https://mcp.brave.com"
    },
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com"
    }
  }
}

```

11.2. Niveles de configuración

Nivel	Ubicación	Prioridad
Global	~/.config/opencode/opencode.json	Baja
Proyecto	opencode.json en raíz	Alta

11.3. Habilitar/deshabilitar por agente

```

{
  "tools": {
    "brave-search*": false
  },
  "agent": {
    "investigador": {
      "tools": {

```

```
    "brave-search*": true
  }
}
}
```

12. Buenas prácticas

- **Limita el alcance** — cada servidor MCP solo debe acceder a lo mínimo necesario
- **No compartas claves** — si un servidor usa API keys, mantenlas fuera del repositorio
- **Documenta los servidores** — incluye un comentario en `mcp.json` explicando qué hace cada uno
- **No cargar todos a la vez** — cada MCP agrega tokens al contexto
- **Usar solo los necesarios** — activa según la tarea
- **Priorizar MCPs de búsqueda** — Fetch, Brave Search, Firecrawl son esenciales

13. Otros servidores MCP disponibles

Servidor	Para qué sirve
<code>server-filesystem</code>	Leer archivos de un directorio seguro
<code>server-sqlite</code>	Consultar una base de datos local
<code>server-github</code>	Leer issues, PRs y repositorios
<code>server-notion</code>	Consultar una base de conocimiento en Notion
<code>server-postgres</code>	Consultar una base de datos PostgreSQL
<code>brave-search</code>	Búsqueda web actualizada
<code>context7</code>	Documentación actualizada de librerías
<code>memory</code>	Memoria persistente entre sesiones

14. Referencias

- [Documentación oficial MCP](#)
- [Lista de servidores MCP](#)
- [Documentación de OpenCode](#)
- [Firecrawl](#)
- [Context7](#)
- [Playwright](#)
- [Quarto MCP](#)

💡 ¿No encuentras lo que buscas?

El ecosistema MCP no para de crecer: más de 19.000 servidores y siguen apareciendo nuevos cada semana. Es muy probable que exista una solución para lo que necesites, o varias.

Puedes pedirle a OpenCode que te ayude a buscar: descríbelle tu necesidad y pídele que liste las opciones disponibles con sus características. Ten en cuenta que algunos MCP tienen la documentación en otros idiomas, lo que puede dificultar su consulta directa. OpenCode puede ayudarte a evaluarlos.

Curso completo disponible en: palazon.github.io/ocCursoAvanzado

Curso OpenCode 102 · Idea original de JA Palazón · Junio 2026